Closer through commonality: Enhancing hypergraph contrastive learning with shared groups

Daeyoung Roh, Donghee Han, Daehee Kim, Keejun Han*, and Mun Yi*

(* Corresponding author)

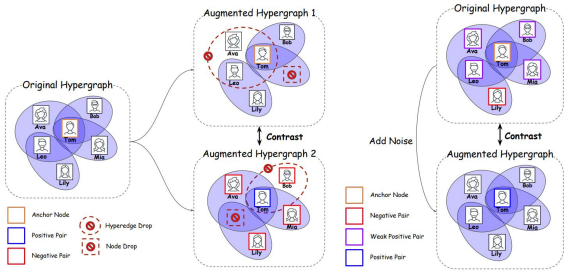
Introduction

- Hypergraphs surpass simple pairwise connections, capturing complex interactions in the real world.

- Despite its potential, research on hypergraph-based learning has been relatively sparse, with even fewer studies exploring contrastive learning, although the possibility of using a hypergraph has recently gained momentum.

- Traditional hypergraph contrastive learning (HGCL) relies on contrastive learning between graphs augmented with random variations in nodes or hyperedges. Consequently, it depends heavily on the embedding of the augmented opposite view.

- We propose **Hy**pergraph **Fi**ne-Grained contrastive learning (HyFi), a novel method that identifies and contrastively learns new positive pairs within the original view without perturbing the topology of the original graph.

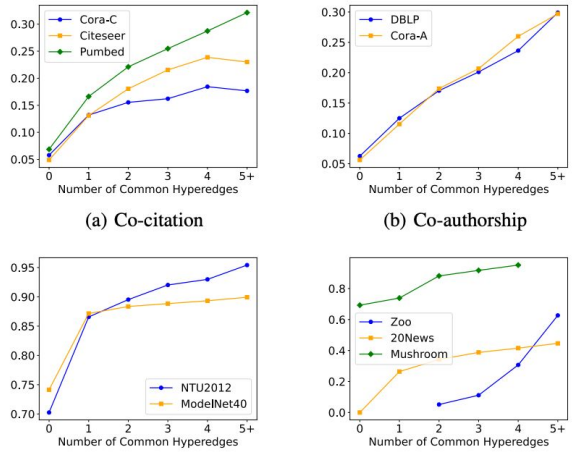


(a) Tranditional HGCL (b) HyFi

Fig. 1: Example of tranditional hypergraph contrastive learning vs. HyFi. In (a), contrastive learning only learns postive or negative pairs between graphs augmented by randomly dropping hyperedges or nodes. In (b), HyFi preserves the topology of the hypergraph by adding noise to the node features, while adding weak postive pair relationships for fine-grained contrastive learning.

Introduction

- We propose a new relation, called a weak positive pair, which shows how group sharing can be exploited in contrastive learning.
- Furthermore, our approach to augmenting the graph by adding noise to the node features to generate positive samples differs from typical graph augmentation techniques that corrupt the original topology and increase model complexity through random modifications.
- As a result, HyFi outperformed 10 supervised and 6 unsupervised baselines in node classification tasks across 10 datasets.
- Furthermore, because HyFi does not modify the topology of the graph, it is widely applicable to a variety of datasets and is more efficient in terms of both time and memory usage compared to traditional HGCL models.



(a) Co-citation (b) Co-authorship

(c) Computer Vision and Graphics (d) Machine Learning Repository

Fig. 2: The average of the cosine similarity of the node features over the number of commonalities between two nodes, where commonality, defined as a common group, means common hyperedges. The higher the number of common hyperedges, the higher the similarity of the two node features.

Contribution

- We propose a novel contrastive learning method, called HyFi. The method exploits the higher-order correlations in hypergraphs represented by commonality to achieve efficient contrast learning that is less dependent on graph augmentation than existing methods.

- We introduce a new relation called weak positive pair, which shows that setting the positive sample to be fine-grained is key to contrastive learning.

- We propose an augmentation scheme that adds noise to the node features in a way that does not preserve the topology of the hypergraph, and show that it is sufficient to generate positive samples that are comparable to various forms of augmentation.

Framework

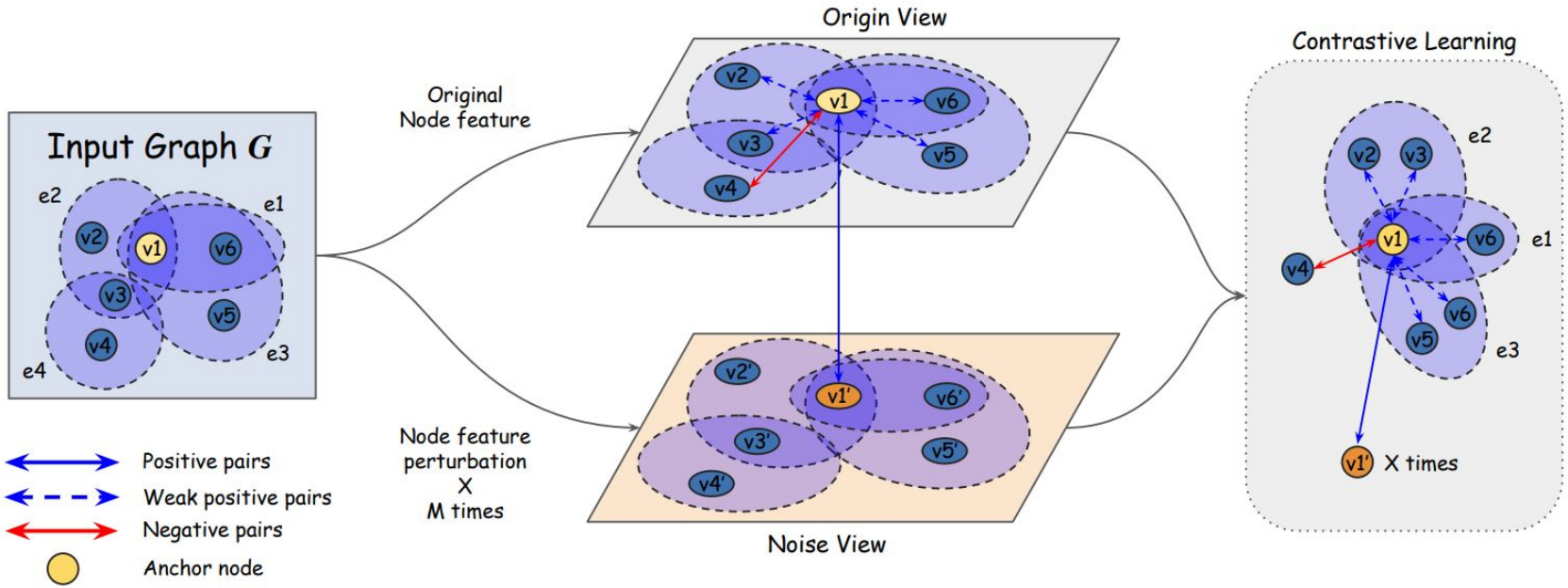


Fig. 3: Illustration of the node-level contrastive loss calculation process in HyFi. HyFi uses both the origin view and the noise view to calculate contrastive loss. In the origin view, there is a negative and a weak postvie pair, and in the noise view there is a postive pair, which is a node in the same position as the anchor node.

Methodology - Node perturbation

- When we have a hypergraph G = (V, E) along with node features X, our approach involves perturbing only X without altering the topology of the hypergraph. The perturbation is executed by adding Gaussian-distributed random noise to the original node features.

$$X' = X + (-1)^X \cdot |\epsilon|, \quad \epsilon \sim \mathcal{N}(0, \sigma^2) \qquad (1)$$

- where X and X′ are the original node feature and the node feature with noise added, respectively. ϵ denotes noise with a Gaussian distribution with a mean of zero and a variance of σ^2 .

Methodology - Hypergraph encoder

- we employ an HGNN[1] model for the hypergraph encoder, denoted by fθ(·).

$$
\begin{aligned}
f_\theta(G, X) &= (P, Q), \\
f_\theta(G, X') &= (P', Q'),
\end{aligned}
\tag{2}
$$

$$
\begin{aligned}
Q^{(k)} &= \sigma\left(D_E^{-1} H^T D_V^{-1/2} P^{(k-1)} \Theta_E^{(k)} + b_E^{(k)}\right), \\
P^{(k)} &= \sigma\left(D_V^{-1/2} H W Q^{(k)} \Theta_V^{(k)} + b_V^{(k)}\right),
\end{aligned}
\tag{3}
$$

$$
\begin{aligned}
Z &:= g_\phi(P) \quad Z' := g_\phi(P'), \\
Y &:= g_\psi(Q) \quad Y' := g_\psi(Q'),
\end{aligned}
\tag{4}
$$

[1] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, Hypergraph neural networks, vol. 33, no. 01, pp. 3558–3565, 2019.

Methodology - Contrastive Loss

- HyFi computes contrastive loss at both the node and edge levels. The core principle of this computation is that nodes sharing hyperedges are considered weak positive pairs in node-level contrastive learning.

- We exploit the unique perspective of each group by performing the contrastive loss computation specifically within each group.

- We compute contrast loss from three perspectives: weak positive relationship, positive relationship, and negative relationship.

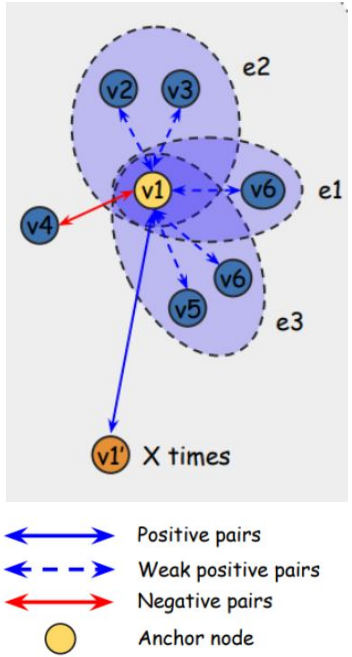
- Finally, the final loss L is calculated by summing the node-Level contrastive loss loss_n and the edge-level contrastive loss loss_e.

Methodology - Contrastive Loss

- Nodes that share the same hyperedge in the original view are considered weak positive pairs. All other nodes, which do not share a hyperedge with the anchor node, are categorized as negative pairs.

- We also assign a weight to the weak positive pair, which represents the probability that a node is a positive pair with the anchor node. This weight is calculated as the ratio of the number of shared hyperedges between two nodes to the total number of hyperedges of the anchor node.

$$w_{i,j}^{pos} = \underbrace{|E_{i,j}|}_{\text{for group unit contrast}} \cdot \underbrace{\frac{|E_{i,j}|}{|E_{i,i}|}}_{\text{postivie probability}},$$



Methodology - Contrastive Loss

- The node-level contrastive loss comprises three components: L_n^{pos} for similarity between original and noisy views, L_n^{weak_pos} for weighted similarity with weak positive pairs, and L_n^{neg} for similarity with negative pairs.

- The final node-level contrastive loss (L_n) is computed as the negative log likelihood of the ratio between positive samples (including both positive and weak positive) and all samples, with temperature parameter τ_n controlling the scale of similarities.

$$\mathcal{L}_n^{pos} = \sum_{m \in M} e^{\mathrm{sim}(z_i, z'_{m,i})/\tau_n},$$

$$\mathcal{L}_n^{weak_pos} = \sum_{j \in N_{z_i}} w_{i,j}^{pos} \cdot e^{\mathrm{sim}(z_i, z_j)}/\tau_n,$$

$$\mathcal{L}_n^{neg} = \sum_{j \notin N_{z_i}} e^{\mathrm{sim}(z_i, z_j)}/\tau_n,$$

$$\mathcal{L}_n(z_i, z'_m) = -\log\left(\frac{\mathcal{L}_n^{pos} + \mathcal{L}_n^{weak_pos}}{\mathcal{L}_n^{pos} + \mathcal{L}_n^{weak_pos} + \mathcal{L}_n^{neg}}\right) \quad (7)$$

Methodology - Contrastive Loss

- The calculation of edge-level contrastive loss is the same process as the calculation of node-level contrastive loss. However, at the edge level, the node and hyperedge relationships at the node level are reversed.

- Finally, the final loss L is calculated by summing the node-Level contrastive loss loss_n and the edge-level contrastive loss loss_e.

$$\mathcal{L} = \sum_{i=1}^{V} \mathcal{L}_n(z_i, z'_m) + \alpha \cdot \sum_{i=1}^{E} \mathcal{L}_e(y_i, y'_m) \tag{8}$$

Research Question

- RQ1 (Generalizability): How does HyFi outperform baselines on node classification tasks?
- RQ2 (Efficiency): How efficient is HyFi regarding GPU memory usage and training time?
- RQ3 (Ablation Study): How do the newly introduced modules in HyFi affect its overall performance?
- RQ4 (Type of Augmentation): How do different augmentation methods affect performance?
- RQ5 (Effect of Positive sample): How does the number of positive samples affect hypergraph contrastive learning?

Experiment Setting & Dataset

- To compare the performance of HyFi, we use a total of 10 datasets.
- HyFi was compared against 10 (semi-)supervised models and 5 unsupervised models. We focused particularly on comparisons with the state-of-the-art model in HGCL, which is TriCL[2].
- For the node classification task, we follow the standard linear evaluation protocol. An encoder is trained in a fully unsupervised manner to generate node representations.
- For all datasets, nodes are randomly assigned to the training, validation, and test sets in proportions of 10%, 10%, and 80%, respectively.
- The experiment was repeated a total of five times, and the average results are reported here.

TABLE II: Statistics for datasets.

Dataset	# Nodes	# Hyperedges	# Classes
Cora-C	1,434	1,579	7
Citeseer	1,458	1,079	6
Pubmed	3,840	7,963	3
Cora-A	2,388	1,072	7
DBLP	41,302	22,363	6
Zoo	101	43	7
20News	16,242	100	4
Mushroom	8,124	298	2
NTU2012	2,012	2,012	67
ModelNet40	12,311	12,311	40

[2] D. Lee and K. Shin, “I’m me, we’re us, and i’m us: Tri-directional contrastive learning on hypergraphs,” vol. 37, no. 7, pp. 8456–8464, 2023.

Experiment - RQ1

TABLE III: Comparison of node classification accuracy. The highest scores in each dataset are highlighted in bold, indicating the best performance by method, while the second highest scores are underlined. The 'A.R.' indicates the average rank across all datasets. '-' indicates that the results are not available in published papers.

	Method	Cora-C	Citeseer	Pubmed	Cora-A	DBLP	Zoo	20News	Mushroom	NTU2012	ModelNet40	A.R.↓
Supervised	MLP	60.32±1.5	62.06±2.3	76.27±1.1	64.05±1.4	81.18±0.2	75.62±9.5	79.19±0.5	99.58±0.3	65.17±2.3	93.75±0.6	11.4
	GCN	77.11±1.8	66.07±2.4	82.63±0.6	73.66±1.3	87.58±0.2	36.79±9.6	-	92.47±0.9	71.17±2.4	91.67±0.2	10.7
	GAT	77.75±2.1	67.62±2.5	81.96±0.7	74.52±1.3	88.59±0.1	36.48±10.0	-	-	70.94±2.6	91.43±0.3	10.0
	HGNN	77.50±1.8	66.16±2.3	83.52±0.7	74.38±1.2	88.32±0.3	78.58±11.1	80.15±0.3	98.59±0.5	72.03±2.4	92.23±0.2	7.1
	HyperConv	76.19±2.1	64.12±2.6	83.42±0.6	73.52±1.0	88.83±0.2	62.53±14.5	79.83±0.4	97.56±0.6	72.62±2.6	91.84±0.1	8.7
	HNHN	76.21±1.7	67.28±2.2	80.97±0.9	74.88±1.6	86.71±1.2	78.89±10.2	79.51±0.4	99.78±0.1	71.45±3.2	92.96±0.2	7.9
	HyperGCN	64.11±7.4	59.92±9.6	78.40±9.2	60.65±9.2	76.59±7.6	40.86±2.1	77.31±6.0	48.26±0.3	46.05±3.9	69.23±2.8	14.1
	HyperSAGE	64.98±5.3	52.43±9.4	79.49±8.7	64.59±4.3	79.63±8.6	40.86±2.1	-	-	-	-	13.7
	UniGCN	77.91±1.9	66.40±1.9	**84.08±0.7**	77.30±1.4	90.31±0.2	72.10±12.1	**80.24±0.4**	98.84±0.5	73.27±2.7	94.62±0.2	5.0
	AllSet	76.21±1.7	67.83±1.8	82.85±0.9	76.94±1.3	90.07±0.3	72.72±11.8	79.90±0.4	99.78±0.1	75.09±2.5	96.85±0.2	5.0
Unsupervised	Random-Init	63.62±3.1	60.44±2.5	67.49±2.2	66.27±2.2	76.57±0.6	78.43±11.0	77.14±0.6	97.40±0.6	74.39±2.6	96.29±0.3	10.9
	Node2vec	70.99±1.4	53.85±1.9	78.75±0.9	58.50±2.1	72.09±0.3	17.02±4.1	63.35±1.7	88.16±0.8	67.72±2.1	84.94±0.4	14.6
	DGI	78.17±1.4	68.81±1.8	80.83±0.6	76.94±1.1	88.00±0.2	36.54±9.7	-	-	72.01±2.5	92.18±0.2	8.2
	GRACE	79.11±1.7	68.65±1.7	80.08±0.7	76.59±1.0	-	37.07±9.3	-	-	70.51±2.4	90.68±0.3	9.4
	S^2-HHGR	78.08±1.7	68.21±1.8	82.13±0.6	78.15±1.1	88.69±0.2	**80.06±11.1**	79.75±0.3	97.15±0.5	73.95±2.4	93.26±0.2	5.7
	TriCL	81.03±1.3	71.97±1.3	83.80±0.6	**82.22±1.1**	**90.93±0.2**	79.47±11.0	79.93±0.2	98.93±0.3	74.63±2.5	97.33±0.1	2.5
	HyFi	**81.48±1.5**	**72.57±1.1**	83.82±0.6	79.33±1.2	90.41±0.2	80.02±10.9	79.76±0.3	**99.79±0.2**	**75.10±2.6**	**97.38±0.1**	**1.9**

Experiment - RQ2

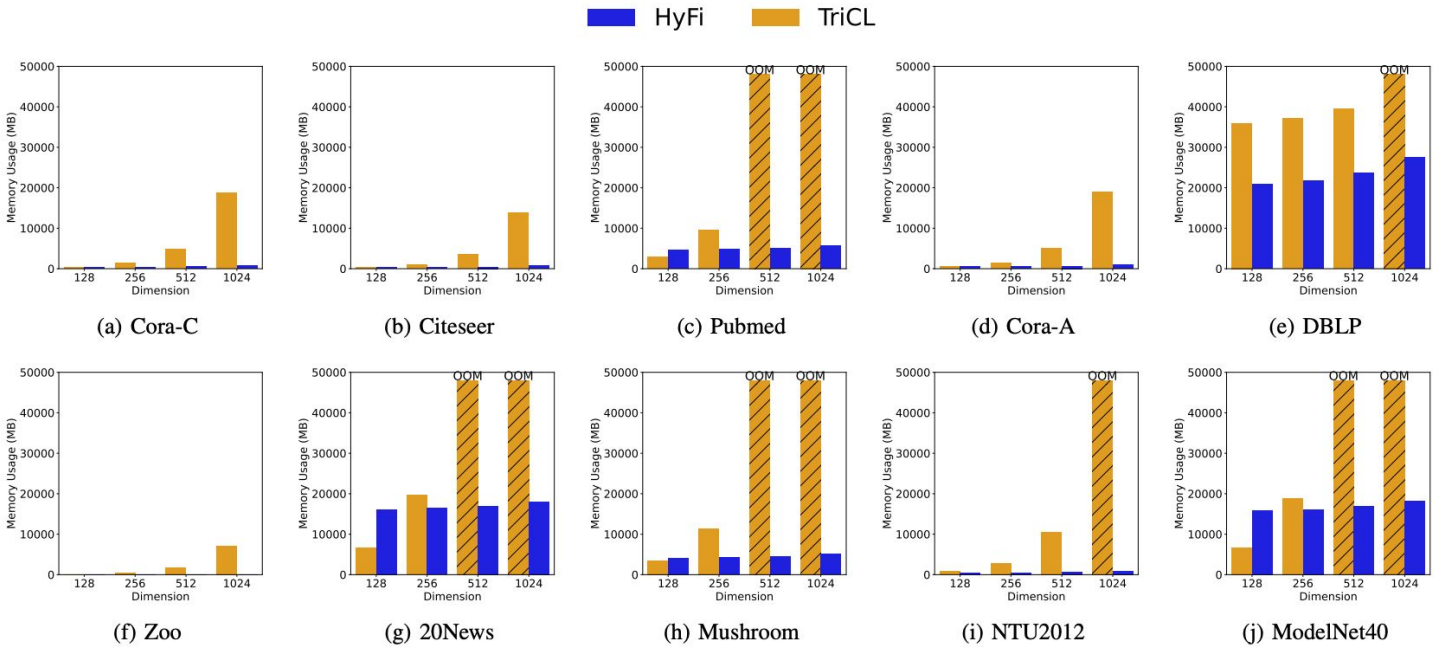


Fig. 4: Illustration of the GPU memory usage comparison between HyFi and TriCL, focusing on how they differ in memory consumption in various dimensions, where dimension denotes the encoder output dimension and the project head output dimension, a bar with a diagonal pattern means 'Out of Memory'.

Experiment - RQ2

TABLE IV: Comparisons of training time.

Dataset	TriCL	HyFi
Cora-C	21.79 s (×1.71)	12.71 s (×1.00)
Citeseer	58.91 s (×3.17)	18.60 s (×1.00)
Pubmed	350.68 s (×5.07)	69.22 s (×1.00)
Cora-A	33.82 s (×1.40)	24.21 s (×1.00)
DBLP	1145.62 s (×2.38)	481.86 s (×1.00)
Zoo	5.11 s (×2.61)	1.96 s (×1.00)
20News	286.86 s (×5.68)	50.47 s (×1.00)
Mushroom	251.69 s (×7.95)	31.67 s (×1.00)
NTU2012	44.08 s (×4.98)	8.86 s (×1.00)
ModelNet	213.52 s (×3.29)	65.04 s (×1.00)

Experiment - RQ3

TABLE V: Ablation study exploring various configurations of the contrastive learning framework 'w/o' stands for 'without'.

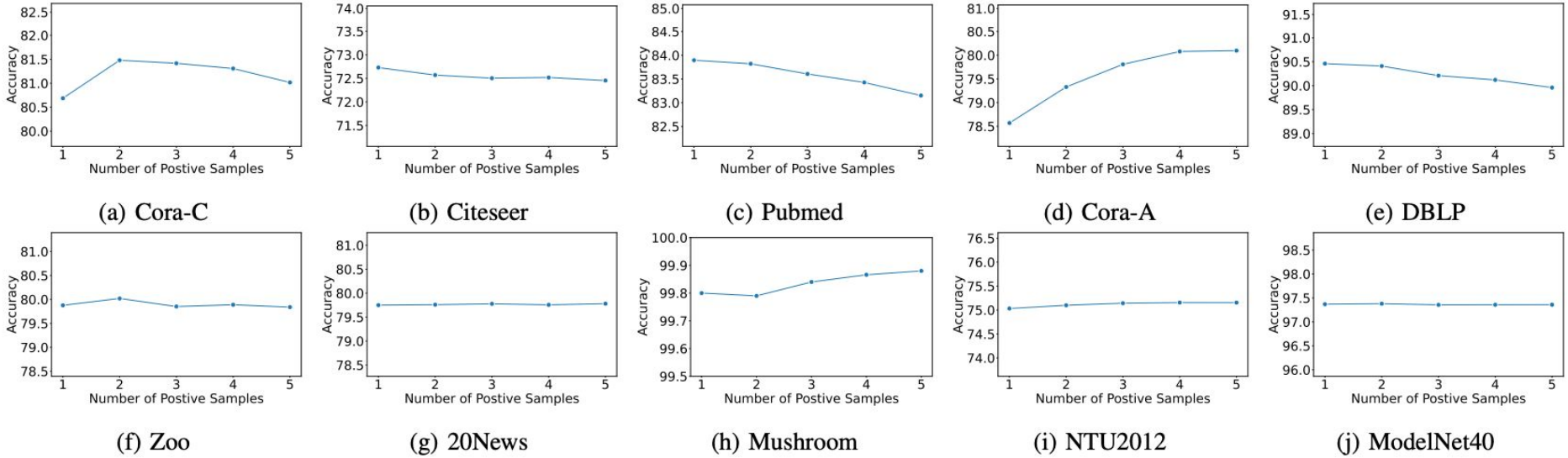
	Cora-C	Citeseer	Pubmed	Cora-A	DBLP	Zoo	20News	Mushroom	NTU2012	ModelNet40	A.R.↓
HyFi	**81.48±1.5**	72.57±1.1	**83.82±0.6**	**79.33±1.2**	90.41±0.2	**80.02±10.9**	**79.76±0.3**	99.79±0.2	**75.10±2.6**	**97.38±0.1**	**1.4**
w/o weak positive pair	70.89±3.6	60.12±3.6	79.91±0.9	75.75±1.1	88.46±0.1	79.79±10.9	79.69±0.3	**99.87±0.1**	74.88±2.6	97.31±0.1	4.2
w/o positive pair	77.96±1.7	71.97±1.1	83.59±0.6	76.65±1.4	87.62±0.6	79.94±11.0	79.74±0.3	99.79±0.1	74.91±2.6	97.35±0.1	3.2
w/o weak weight	80.44±1.3	**72.61±1.1**	83.79±0.7	78.44±1.1	**90.54±0.1**	79.77±10.9	**79.76±0.3**	99.76±0.2	74.52±2.5	97.32±0.1	2.9
w/o edge loss	81.25±1.4	72.42±1.2	83.77±0.7	79.04±1.1	90.31±0.2	79.79±11.0	79.74±0.3	99.79±0.2	74.94±2.6	97.26±0.1	2.8

Experiment - RQ4

TABLE VI: Performance of different augmentation method on various datasets.

Augmentation Type	Method	Cora-C	Citeseer	Pubmed	Cora-A	DBLP	Zoo	20News	Mushroom	NTU2012	ModelNet40	A.R.↓
Add Noise	Gaussian Noise	**81.48±1.5**	**72.57±1.1**	83.82±0.6	**79.33±1.2**	**90.41±0.2**	**80.02±10.9**	**79.76±0.3**	99.79±0.2	**75.10±2.6**	**97.38±0.1**	**1.4**
	Uniform Noise	81.40±1.2	72.51±1.1	**84.03±0.7**	77.61±1.2	90.37±0.2	79.65±10.8	**79.76±0.3**	99.79±0.1	74.52±2.5	97.32±0.1	2.9
	Bernoulli Noise	76.78±1.5	71.17±1.3	82.54±0.7	76.78±1.2	90.01±0.2	79.60±10.8	**79.76±0.3**	99.78±0.1	74.40±2.5	97.29±0.1	5.2
Drop	Incidence	78.81±1.5	72.22±1.2	83.59±0.6	77.54±1.3	89.12±0.4	79.91±10.9	79.73±0.3	99.82±0.1	75.05±2.6	97.37±0.1	3.5
	Node	78.89±1.5	72.25±1.2	83.58±0.6	77.57±1.3	89.05±0.4	79.90±10.9	79.73±0.3	99.82±0.1	75.08±2.6	**97.38±0.1**	3.1
	Hyperedge	78.71±1.5	72.25±1.1	83.58±0.6	77.30±1.4	88.86±0.4	79.98±11.0	79.73±0.3	**99.83±0.1**	75.04±2.6	97.37±0.1	3.7

Experiment - RQ5



(a) Cora-C (b) Citeseer (c) Pubmed (d) Cora-A (e) DBLP

(f) Zoo (g) 20News (h) Mushroom (i) NTU2012 (j) ModelNet40

Fig. 5: Illustration of the performance of node classification as the number of positive samples. The number of noise views is equal to the number of positive samples in the contrastive learning.

Conclusion

- This study has successfully demonstrated the efficacy of the HyFi model in hypergraph-based contrastive learning, demonstrating its ability to capture the nuanced relationships inherent in hypergraph results in higher quality embeddings, and superior performance in node classification task across a variety of datasets.

- Our novel augmentation technique developed for hypergraphs preserves the original topology while adding noise to node features, ensuring the integrity of the hypergraph structure.

- Furthermore, the contrastive learning method introduced in this study effectively utilizes the homophily characteristic of hypergraphs, providing a more effective approach than existing HGCL methods.

- Also, the proposed HyFi model stands out for its efficiency in processing speed and memory usage, making it an ideal solution for various applications.

Thank you